

COURTFLOW-GS: Online Multi-View Dynamic Gaussian Reconstruction with Fused MLS Tracking

Shangar Muhunthan

September 28, 2026

Abstract

COURTFLOW-GS turns synchronized videos from cameras around a basketball court into a 3D scene that can be viewed from a new camera angle. It first builds one clean, static “starting scene” from frame 0. It then follows each player through the video by moving small sets of control points, and occasionally improves visual detail. On one RTX PRO 6000 it reconstructs all 700 frames from 12 of the 36 cameras in about two hours, and the 24 cameras it never sees score 20.2 dB PSNR on average over the whole video. The result is a 3D reconstruction, novel-view videos, and a saved Gaussian model that can be rendered from any camera without retraining.

1 Problem and data

The data contains 36 synchronized 4K cameras placed in a ring around the court. Their positions, orientations, and lenses have already been calibrated, so the system knows exactly where every camera is. We train with every third camera (views 0, 3, ..., 33) and keep the remaining 24 cameras aside for an honest test from viewpoints the model did not see during training. The camera calibration is fixed throughout the method.

The input adapter reads the 36 videos and the COLMAP calibration (`cameras.txt` and `images.txt`) and creates an undistorted working copy for training. By default this copy is half resolution (1920×1080), which keeps the camera geometry while making the many repeated training renders cheap; the same pipeline also runs at the native 3840×2160 .

2 Existing methods and the 12-view problem

How other methods handle moving scenes. A Gaussian splatting scene is built from millions of small, coloured blobs called Gaussians. To show motion, a method has to decide how those blobs move from one video frame to the next. Most recent methods let every blob move on its own:

- **Dynamic 3D Gaussians** [1] stores a new position and rotation for every blob at every frame, with a penalty that keeps neighbouring blobs moving together.
- **3DGStream** [2] trains a small neural network for each frame that says how every blob moves.
- **QUEEN** [3] stores a compressed per-blob change for each frame.
- **ClipGStream** [4] splits the video into short clips and learns the motion of each clip together.
- **SC-GS** [5] is closest to our method: a few control points move the blobs. But it learns the motion with a neural network trained on the whole video at once, and it was built for dense or single-camera video.

All of these were tested with many more cameras than this assignment gives us (Table 1). The closest test is ClipGStream’s “Long 360” scene: a ring of 36 4K cameras, just like ours, but it trains on 32 of them. We may train on only 12. With 12 cameras, neighbouring cameras are 30° apart, and a distant player may be clearly visible in only a few of them. When every blob can move freely, the method can match those few images exactly while putting blobs in the wrong 3D place. The result looks right from the training cameras and wrong from any new camera.

Table 1: The settings each method was published with. Numbers come from the papers, which use different datasets and GPUs, so this shows what each method was built for; it is not a direct speed or quality comparison.

Method	Training cameras	How blobs move	Reported training time
Dynamic 3D Gaussians	27 of 31 (dome)	each blob, every frame	2 h for 150 frames (RTX 3090)
3DGStream	20 of 21, or 13 of 14	small network per frame	12 s per frame, plus 23 min for the first frame (RTX 3090)
QUEEN	20 of 21	compressed change per blob	6 s per frame (A100)
ClipGStream	32 of 36 (4K ring)	learned per short clip	about 30 min per video
SC-GS	dense or one camera	control points + network	trained on the whole video
COURTFLOW-GS (ours)	12 of 36 (4K ring)	control points per player	Table 8

We tried ClipGStream on this exact data. Before building COURTFLOW-GS, we ran ClipGStream on this capture with the same 12 training cameras and the same 24 test cameras (on an RTX 3080, at 960×540 , on the first 10 frames). Table 2 shows the problem described above. On the training cameras it scored 26–28 dB PSNR (higher is better), but on the 24 test cameras only 17.2–17.9 dB. Training for longer made the test score *worse* (17.88 dB after 6000 steps, 17.53 dB after 40000) even though the training score went up: the model was memorizing the 12 views rather than learning the 3D scene. It also took about 13.5 s per frame on the 3080, after a 7.2 minute start-up.

Table 2: ClipGStream on this capture. **Setup:** RTX 3080 (12 GB); 960×540 (the 4K capture quartered); 12 training cameras; scores averaged over the 24 test cameras on the first 10 video frames. PSNR and SSIM: higher is better. LPIPS: lower is better.

Run	Training PSNR	Test PSNR	Test SSIM	Test LPIPS
6000 steps	25.96	17.88	0.543	0.308
40000 steps	28.22	17.53	0.522	0.320
40000 steps, focused on the court	–	17.22	0.498	0.342
Full video run, first clip	–	17.93	0.546	0.306

What COURTFLOW-GS does differently. COURTFLOW-GS was designed for few cameras from the start:

- **Far fewer things to solve for.** A player’s blobs cannot move freely. They follow 1024 control points, and each player first moves as a whole, then adds small movements for arms and legs, while keeping its shape. Twelve cameras are enough to pin down a few thousand control points; they are not enough to pin down millions of free blobs.
- **Shape comes from more than colour.** With few cameras, many wrong 3D shapes can produce the right colours. COURTFLOW-GS first builds each player’s rough 3D shape from where the player appears in every camera (the “visual hull”) and from matched points between cameras. It also randomly hides blobs during training (DropGaussian [6]) so no single blob can memorize one view.

- **Each player is kept separate.** Every player has their own control points, so one player can never drag a teammate or the floor along. When the automatic masks mix two players up, the mistake is fixed by checking which player is where in 3D.
- **The still parts are done once.** Players cover only about 4% of each image; the rest is court and crowd that never moves. Other methods re-fit that area every frame. COURTFLOW-GS builds it once at frame 0 and afterwards only updates the players. This makes it both faster (less work per frame) and better (the court cannot slowly drift out of place).

The catch is a slower start: building the first frame carefully (Stage A) takes longer than a plain 3D Gaussian start. But that happens once per scene; after it, each frame takes a few seconds.

3 Methodology

Before doing anything else, every image is undistorted. This removes lens bending, so a straight camera ray in the mathematics corresponds to a straight ray in the image. In the notation used later, camera c is $P_c = K_c[R_c | t_c]$, with world-to-camera conversion $x_c = R_c x + t_c$. Readers do not need this equation to follow the method: it only says that the known calibration projects a 3D point into each image. Camera poses, lens parameters, and per-camera brightness corrections are kept fixed.

3.1 Frame-0 Reconstruction

Masks and 3D identities: Photometric reconstruction alone is too ambiguous for the moving players. Each player occupies a small portion of a frame, shares colour and texture with teammates, and is frequently overlapped by another player or various structures / elements in the scene. Foreground masks tell the system both “which pixels belong to this person for geometry matching” and “where this person is allowed to appear when rendered.” This prevents background splats from being absorbed into players and prevents players from growing fuzzy opaque halos onto the court. It also tells the optimizer which Gaussian/control-point group should explain each pixel. This prevents a global reconstruction from improving RGB error by assigning one player’s appearance or motion to another player or to the background.

Simply running a detector in each camera is not enough: “person 4” in one image has no reason to be the same physical person as “person 4” in another. We first tried matching 2D mask centres and colours between cameras. With cameras about 30° apart, that split 68 masks into 60 supposed people. Full-frame MAST3R matching was also too coarse for small players and made unreliable star-shaped rays. Instead, CourtFlow-GS establishes identity *in 3D first*.

Faster R-CNN proposes person boxes (score ≥ 0.5), and SAM2 turns each box into a precise foreground mask. We use agreement between those masks to make rough 3D blobs, where each blob is a likely physical player. We project each blob back into every camera and match it to the local SAM2 mask. When a mask is missing, the projected blob tells SAM2 where to look again. In this way, noisy per-camera masks become one consistent player ID across the ring.

Concretely, we cover the court with a 2.4 m-high box that extends 1 m beyond the court lines, and divide it into small cubes called voxels. A voxel is kept only if enough cameras see it inside a person mask: at least $\max(4, n_{\text{visible}} - 2)$ cameras. Put simply, several cameras must agree, but up to two are allowed to disagree because a player can be hidden. We discard blobs that are too small, too short, or floating above the floor. A blob wide enough to contain two people is split on the court floor.

For a missing camera mask, we accept the new SAM2 result only if its colour is similar to that same 3D player in the other cameras. If it still looks wrong, we mark that player as *unknown* in that camera, not as background. This is important: a hidden or missed player should not teach the model that the court is visible there. Finally, if two accepted masks overlap, the closer player owns the pixel. The result is one label image L_c

per camera in which every pixel is either background or exactly one player. Later, this lets every player have separate Gaussian splats and separate motion controls.

Occlusion-aware visual hull. A visual hull is a rough 3D body made by intersecting the player silhouettes from several cameras. The usual rule says to remove a voxel if even one camera does not see the player there. That works for a clean object on a turntable, but fails in basketball: another player, the hoop support, or padding can hide a real arm, leg, or head. We saw those missing body parts with a strict hull.

The opposite rule (forgiving any missing view) was also bad. One allowed miss made player hulls about $2.5\times$ too large. Our compromise asks one simple question: *is there measured evidence that something closer to this camera was blocking the player?* We forgive a missing pixel only when the answer is yes. This is what “occlusion-aware” means here.

For each player, we test a fine 1 cm grid inside the rough 3D blob, with a small margin. Camera c votes against a candidate voxel x only when the voxel is visible in that camera, the player has a trusted label there, the projected pixel is not labelled as that player, and nothing nearer explains why. A nearer explanation can be another labelled player or a measured background surface (such as the hoop support) at least 30 cm in front. The background depth map D_c comes from reliable full-frame MAST3R background points that agree in at least two additional cameras. We retain the nearest depth at each image location and ignore floor points within 0.5 m of the court.

We keep x when at least two cameras see it as the player and no camera has an *unexplained* reason to reject it. Voxels below the floor are then removed. A camera where the player is unknown casts no vote. Thus the method does not keep geometry merely because occlusion might be possible; it keeps it only when a nearer player or surface actually supports that explanation.

Masked crop geometry and Gaussian seeds. Feature matching across a full image works for the court, but a distant player may occupy only about 6×25 pixels once MAST3R shrinks the whole frame to 512 pixels. Our full-frame player matches made a radial “star” of unreliable 3D rays. The masks solve this by giving the player the full matching resolution: for nearby camera pairs, we crop a padded square around the same 3D player and resize that crop to 512×512 . MAST3R searches only inside the source player mask.

After mapping matches back into the original camera images, we retain only confident matches that are in front of the cameras, reproject accurately, use cameras far enough apart for useful depth, lie inside a slightly expanded player hull, and agree with at least one more labelled camera. This removes the court and background rays that slipped through earlier mask-only tests.

The retained matches are accurate surface seeds. Hull voxels fill only textureless gaps more than two hull voxels away from a seed. We colour points only from cameras that agree with the label and can actually see the point, remove outliers, merge nearby player points at 5 mm and background points at 10 cm, and turn the result into oriented, elongated Gaussian splats. Each Gaussian stores position, shape, opacity, and simple view-dependent colour (degree-1 spherical harmonics). Players train for 7000 iterations (four cameras per step) and background for 10000 (two cameras per step), using

$$\mathcal{L}_{\text{image}} = \|I - \hat{I}\|_1 + 0.2\text{D-SSIM} + 0.1\mathcal{L}_{\text{alpha}} + 0.05\mathcal{L}_{\text{depth}}.$$

This loss asks the rendered image to match the real image, its structure, its silhouette, and its initial depth. The depth term fades out halfway through training, after it has helped establish the coarse shape. Starting at iteration 500, we add detail where needed every 100 iterations until halfway through training. We remove nearly transparent splats and splats that land outside an expanded player mask in two cameras.

3.2 Group-aware control-point MLS

Control Groups After frame 0, the task is to move the same Gaussian splats as players move. Rather than moving millions of splats independently, we place a much smaller set of control points on each player. Moving

a control point moves nearby splats smoothly. This is called moving-least-squares (MLS) deformation.

A single global control graph is reasonable for one connected object, but unsafe in a crowded game. Controls on two nearby players can otherwise explain the same image error, causing one player to pull another player’s splats or the background. CourtFlow-GS treats every identity as a separate motion group. This keeps the useful smooth motion of MLS while forbidding cross-player influence.

Every dynamic Gaussian belongs to exactly one person; the ball, when enabled, is another group, and background is group 0. We choose controls spread evenly over each object: 1024 per person, 16 for the ball, and 1000 for background. Each Gaussian is permanently connected to its eight nearest controls from the *same* group. Closer controls get more influence through the fixed radial weights

$$w_{ij} = \frac{\exp(-\|x_i - p_j\|^2 / (2\sigma_j^2))}{\sum_{\ell=1}^8 \exp(-\|x_i - p_\ell\|^2 / (2\sigma_\ell^2))},$$

where σ_j is the average distance to control j ’s three nearest controls. The six nearest same-group controls form the shape-preserving ARAP graph. In plain terms, a player’s splats follow that player’s nearby controls, never a teammate’s or the court’s.

Shared Translation + Local Motion Trying to move every control independently made weakly observed controls lag or drift. By frame 10 that version reached 0.60 mask overlap (IoU), compared with 0.71 when each player first gets one shared translation and then small local offsets. The shared translation handles running across the court; the local offsets handle arms, legs, and other articulation. A rigid local MLS transform and the ARAP shape loss keep neighbouring splats from stretching apart.

At frame f , controls begin from the previous frame’s velocity. We optimize one shared translation T_k per moving group and small local offsets δ_j for 50 Adam iterations, using four cameras per iteration. Their learning rates are 0.020 m and 0.003 m. Let $t_j = T_{g(j)} + \delta_j$ and $p'_j = p_j + t_j$. For Gaussian i , the fixed weights above blend nearby original controls p_j and moved controls p'_j to form a best local rotation R_i :

$$\bar{p}_i = \sum_j w_{ij} p_j, \quad \bar{q}_i = \sum_j w_{ij} p'_j, \quad M_i = \sum_j w_{ij} (p_j - \bar{p}_i)(p'_j - \bar{q}_i)^T.$$

With $M_i = USV^T$, set $R_i = VU^T$, flipping the last column of V if $\det(VU^T) < 0$. The final Gaussian position is $x'_i = R_i(x_i - \bar{p}_i) + \bar{q}_i$; its orientation and simple colour direction are rotated too. A fused CUDA operation performs this transform and its backward pass. The reported run does not separately optimize control rotations.

Every fifth frame, we render where each player is expected, prompt SAM2 with that box and centre, and assign overlapping pixels to the strongest rendered player. We triangulate the 2D mask centres from several cameras into a rough 3D player centre. This is a *motion anchor*: it gives the shared translation a reliable hint about where a player moved, especially after fast motion or occlusion. If enough cameras see the player, a robust Huber anchor of scale 0.3 m and weight 0.1 is used. Between those frames, no made-up mask or anchor is used. The deformation objective is

$$\mathcal{L} = \mathcal{L}_{\text{image}} + 0.3\mathcal{L}_{\text{BCE}}^{\text{instance}} + \mathcal{L}_{\text{ARAP}} + 0.01\mathcal{L}_{\text{accel}} + \mathcal{L}_{\text{anchor}},$$

Here the image term matches appearance, BCE matches each player’s rendered mask, ARAP preserves local shape, acceleration discourages sudden jitter, and the anchor discourages global drift. Background controls stay frozen while the moving splats are rendered over a cached background.

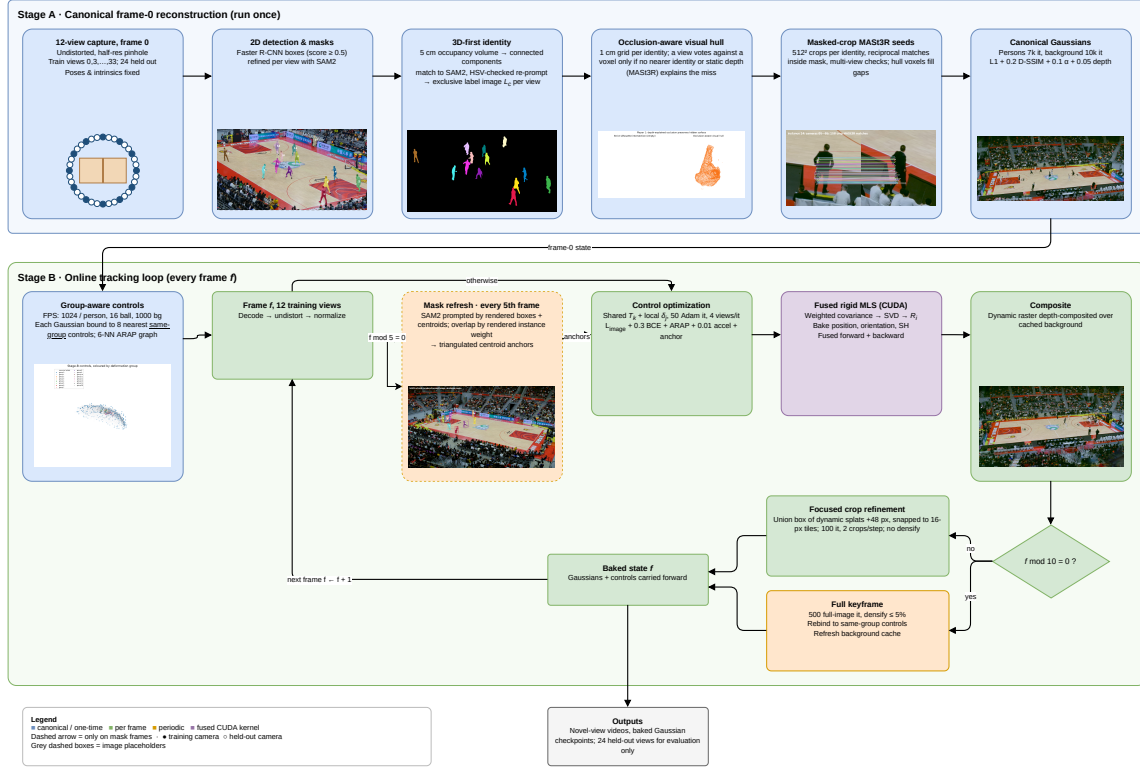


Figure 1: CourtFlow-GS pipeline. The editable diagram source is distributed with the report as `courtflow_gs_pipeline.drawio`.

3.3 Focused refinement

MLS provides stable, smooth motion, but deliberately does not change a Gaussian’s colour, size, or amount of detail. Updating the full image after every video frame would be wasteful: players cover less than 1% of the court image, so most compute would repeatedly optimize the static floor. We therefore refine only the moving-player crops most frames, and use occasional full-image keyframes to correct drift and add a limited amount of new detail.

After every deformation, we save (“bake”) the moved Gaussian and control state for that frame. On a normal frame, all moving splats are projected into each training camera and surrounded by one box with 48 pixels of padding. The box is aligned to the rasterizer’s 16-pixel tiles, and the camera principal point is shifted so rendering the crop is exactly equivalent to rendering that part of the full image. We run 100 refinement iterations on two crops per step. No splats are added or removed here, so their control bindings stay stable.

Every tenth frame is a keyframe. It instead runs 500 full-image iterations, can add splats from iteration 50 every 50 iterations until 60% of the schedule, and limits growth to 5% per keyframe and to 1.5 times the frame-0 size in total (Figure 2 shows why the total cap matters). It then reconnects each Gaussian to its eight nearest same-group controls and refreshes the cached background. Held-out cameras are never included in any training loss.

3.4 Pseudocode

Algorithm 1 summarizes the whole process. The helper names are intentionally literal: first build the frame-0 scene, then for each video frame estimate motion, bake that motion, and improve either player crops or a periodic full keyframe.

Algorithm 1 CourtFlow-GS online reconstruction

Require: Frame-0 images, synchronized training videos, fixed calibration

```
1:  $(state, controls) \leftarrow \text{BUILDCANONICALSCENE}(frame_0, calibration)$ 
2: for each  $f$  in the video sequence do
3:    $images \leftarrow \text{DECODEUNDISTORTNORMALIZE}(trainingVideos, f)$ 
4:   if  $f \bmod 5 = 0$  then
5:      $masks \leftarrow \text{REPAIRMULTIVIEWSAM2IDENTITIES}(images, state.dynamic)$ 
6:      $anchors \leftarrow \text{TRIANGULATEMASKCENTROIDS}(masks)$ 
7:   else
8:      $masks, anchors \leftarrow \emptyset, \emptyset$ 
9:   end if
10:   $offsets \leftarrow \text{OPTIMIZECONTROLS}(L_1 + \text{D-SSIM}, masks, anchors, \text{ARAP})$ 
11:   $state \leftarrow \text{BAKEFUSEDRIGIDMLS}(state, controls, offsets)$ 
12:  if  $f \bmod 10 \neq 0$  then
13:     $state \leftarrow \text{REFINEDYNAMICCROPS}(state, images, 100)$ 
14:  else
15:     $state \leftarrow \text{REFINEFULLIMAGES}(state, images, 500)$ 
16:     $controls \leftarrow \text{REBIND}(state, controls); \text{CACHEBACKGROUND}(state)$ 
17:  end if
18:   $\text{SAVEDIAGNOSTICS}(state)$ 
19: end for
```

4 Execution Walkthrough

The previous section explains *why* each part of the method exists. This section follows one complete run from the raw capture to the final videos: which program runs, what it reads, what it writes, and where time is spent. Everything is started by a single command,

```
python -m ring_init.e2e --scene basketball --dataset-dir <capture> \
    --config config.local.json --out-dir <out> --frames 0:700
```

where the capture directory holds the 36 videos (cameras/view_XXX.mp4) and the COLMAP calibration (calibration/). The local configuration only names the SAM2 checkpoint; all other settings are the defaults, and the exact resolved configuration is written to `e2e_config.json` in the output directory before any work starts. Every step writes a completion marker (`done.json`) next to its output, so an interrupted run resumes where it stopped and `--force` step recomputes a single step. The times below were measured on the RTX PRO 6000.

4.1 Preparing the input (17 s)

The capture adapter reads the calibration for all 36 cameras, decodes frame 0 of every video with `ffmpeg` (Lanczos resampling from 3840×2160 to the 1920×1080 working resolution), and removes lens distortion with `OpenCV` so every camera becomes an ideal pinhole camera. Every third view is copied as the 12 training images `cam_00...cam_11`; all 36 undistorted frame-0 images are kept for held-out evaluation. The adapter also writes the 12-camera and 36-camera calibration files used by the rest of the pipeline.

4.2 Stage A: building the frame-0 scene (≈ 9 min)

Stage A runs the steps of Table 3 in order. Each step caches its result on disk, and later steps only read those files, so the steps are independent programs joined by files.

Table 3: Stage A steps and their wall-clock times. **Setup:** RTX PRO 6000 (96 GB); frame 0 of the 12 training cameras at 1920×1080 ; timed during the 700-frame run.

Step	What runs	Writes	Time
calib	Loads the 12 training cameras and images; draws the camera layout.	calib/	2.1 s
match_full	MASt3R matches neighbouring ring cameras on whole frames (512 px).	3D points	33.4 s
floor	RANSAC fits the court plane and its extent; fixes metric scale.	floor plane	0.3 s
persons	Faster R-CNN person boxes, then SAM2 masks, in every training camera.	per-camera masks	11.7 s
instances	Carves the court volume with all masks; 3D blobs become player IDs; re-prompts SAM2 for missing views.	label images L_c	16.7 s
hulls	Occlusion-aware visual hull for every player.	voxel hulls	4.3 s
match_crops	MASt3R on masked player crops for nearby camera pairs.	player points	46.9 s
fuse	Merges crop matches and hull fill into coloured seed clouds.	seed .ply	5.6 s
train	Optimizes each player’s Gaussians, then the background with players composited in front.	Gaussian models	406.9 s
export	Copies the trained models into a canonical package with a manifest.	canonical/	<1 s
eval	Renders frame 0 for all 24 held-out cameras and scores it.	eval/	–

Training is the only substantial cost. The detector, SAM2, and MASt3R networks are freed from GPU memory once their steps finish, so none of them competes with Gaussian training for memory.

4.3 Stage B, part 1: decoding the sequence

Before tracking starts, Stage B prepares all per-frame data. Six worker processes each stream one camera’s video through ffmpeg and apply exactly the frame-0 preprocessing: the same resampling, the same precomputed undistortion map, and the same per-camera colour correction. The 12 training cameras are decoded at every frame (8400 images for 700 frames). The 24 held-out cameras are decoded only at every tenth frame, because they are used only for scoring. Frames are stored as lightly compressed PNG files, and already decoded frames are skipped on a rerun.

SAM2 then runs once per training camera in video mode. It is prompted with that camera’s frame-0 player labels from Stage A and follows every player through the whole sequence. These raw masks are not trusted as they are: SAM2 treats each camera separately and can swap two players in one camera but not another. Identities are therefore repaired in 3D during tracking (below). Finally, the ball is detected and triangulated at frame 0, tracked causally through the sequence with a gravity and bounce model, and added to the canonical scene as its own small group of Gaussians.

4.4 Stage B, part 2: the per-frame loop

The canonical Gaussians are loaded onto the GPU once and stay there for the whole sequence. Control points are placed on every group (1024 per player, 16 on the ball, 1000 on the background), and each Gaussian is bound to its eight nearest same-group controls. Then, for every frame f :

1. **Load.** A background thread has already read frame f ’s 12 training images from disk while the GPU was busy with frame $f - 1$. The images are downsampled to the tracking resolution (960×540) and uploaded; the full-resolution copies are kept for refinement.
2. **Masks and anchors (every fifth frame).** The SAM2 masks for frame f are read, their identities are rebuilt in 3D exactly as in Stage A’s `instances` step, and the repaired masks are matched to the tracked

players by position and jersey colour. Triangulating each player’s mask centre gives a motion anchor. On other frames, no masks or anchors are used.

3. **Deform.** Fifty Adam iterations update each player’s shared translation and local control offsets; the fused CUDA MLS operation moves the Gaussians. Background controls stay frozen, and the moving players are rendered over a cached background image.
4. **Improve detail.** On nine frames out of ten, 100 iterations refine only the crops around moving players. Every tenth frame is a keyframe. First, a health check compares each player’s rendered footprint with the repaired masks; a player whose overlap has fallen below 0.10 IoU is re-initialized from its canonical shape at its anchor. Then 500 full-image iterations run with limited densification (at most 5% more Gaussians per keyframe, and never more than 1.5 times the frame-0 total), the Gaussians are re-bound to their controls, and the background cache is refreshed.
5. **Record.** The current state is rendered for held-out views 13 and 26 and for the orbit camera and streamed to ffmpeg, so each video frame shows the scene at that instant. Every tenth frame, and at the last frame, all 24 held-out cameras are rendered and scored (PSNR, SSIM, LPIPS, and player-region PSNR); the training views also report mask IoU. Every 25 frames, the full Gaussian state is saved as a PLY checkpoint and the control trajectories and metrics are flushed to disk.

After the last frame, the final state is saved and per-stage timings and quality statistics are summarized in `summary.json`. The held-out view-13 video and the orbit video are then published to `final_renders/` with an `animated_` prefix, marking them as renders of the moving scene rather than of one baked checkpoint.

4.5 What the run leaves on disk

The output directory keeps every intermediate product, so any step can be inspected or rerun:

```
<out>/prepared_input/basketball/  undistorted frame-0 images and calibration
<out>/basketball/
  e2e_config.json, timings.json  resolved settings and step times
  persons/ instances/ hulls/ ... Stage A intermediate results
  canonical/                    editable frame-0 Gaussian scene + manifest
  eval/                         frame-0 held-out scores and images
  frames/ eval_frames/          decoded training and held-out frames
  video_masks/ ball/           SAM2 masks and the ball trajectory
  stage_b_v2/
    metrics.json summary.json  per-frame metrics and the run summary
    control_positions.npz      control-point trajectories for all frames
    ply/frame_NNNNNN/         Gaussian checkpoints every 25 frames
    videos/ final_renders/     held-out and orbit videos
```

5 Results

How quality is measured. Every score below comes from the 24 test cameras, which the model never trains on. We use four common image scores. **PSNR** (in dB) compares pixel colours; higher is better, and about 1 dB is a visible difference. **SSIM** (0 to 1) compares local structure such as edges; higher is better. **LPIPS** (0 to 1) is a learned measure of how different two images look to a person; lower is better. **Player PSNR** is PSNR measured only on the pixels around players, because players cover only about 4% of each image and a full-image score is dominated by the static court and crowd. The test cameras are scored at every tenth video frame and at the last frame (69 frames). Each frame’s score is the average over the 24 test cameras; the table then gives the average over those frames $\pm$ how much it varies from frame to frame.

Table 4: Final quality on the 24 test cameras. **Setup:** RTX PRO 6000 (96 GB); training and test images at 1920×1080 (the 4K capture halved, then undistorted); all 700 frames. The frame-0 row scores the starting scene alone; its player score uses a different player-region definition, so it is not shown.

Result	PSNR	SSIM	LPIPS	Player PSNR
Frame-0 scene (Stage A)	20.08	0.679	0.229	–
Whole video, final method	20.19 ± 0.52	0.681 ± 0.017	0.238 ± 0.014	22.75 ± 1.71
first half (frames 0–349)	20.65	–	–	23.85
second half (frames 350–699)	19.73	–	–	21.69
Whole video, before the growth fix	19.34 ± 1.49	0.675 ± 0.035	0.243 ± 0.031	23.70 ± 1.64

For comparison, ClipGStream reached 17.2–17.9 dB on the same 24 test cameras (Table 2), and it did so at a lower resolution (960×540), which makes PSNR easier, and only on the first 10 frames. COURTFlow-GS holds 20.19 dB on average over all 700 frames at 1920×1080 .

5.1 A failure that only appears late in the video

Development runs covered at most 300 frames. The first full 700-frame run on the RTX PRO 6000 revealed a problem that shorter runs could not show (Figure 2, orange). Each keyframe may add up to 5% more Gaussians to add detail. That rule compounds: after 70 keyframes the scene had grown from 0.74 million to 6.86 million Gaussians. The extra Gaussians were fitted to the 12 training cameras only, so from the test cameras they appeared as dark floaters and smears across the court and the stands nearest the camera (Figure 3). Test PSNR fell from 20.8 dB to 16.1 dB.

We tested three fixes on the first 300 frames before choosing one. Freezing the background at keyframes was about 2 dB *worse* from frame 50 onward: keyframes are genuinely useful for the background, for example to fill in court that was hidden behind a player at frame 0. Capping the total size worked. The final method still lets each keyframe add up to 5%, but never lets the scene grow beyond 1.5 times its frame-0 size. With the cap, test PSNR ends at 19.3 dB instead of 16.1 dB (Figure 2, blue).

The fix improved speed and quality together, which is the trade-off the assignment asks about. Tracking became faster (3.71 instead of 4.87 s per frame), peak GPU memory fell from 12.1 to 7.2 GB, the final model shrank from 714 MB to 114 MB, and the test PSNR over the whole video rose by 0.85 dB. The cost is about 1 dB of player PSNR: with a fixed budget, players receive fewer new Gaussians for extra detail.

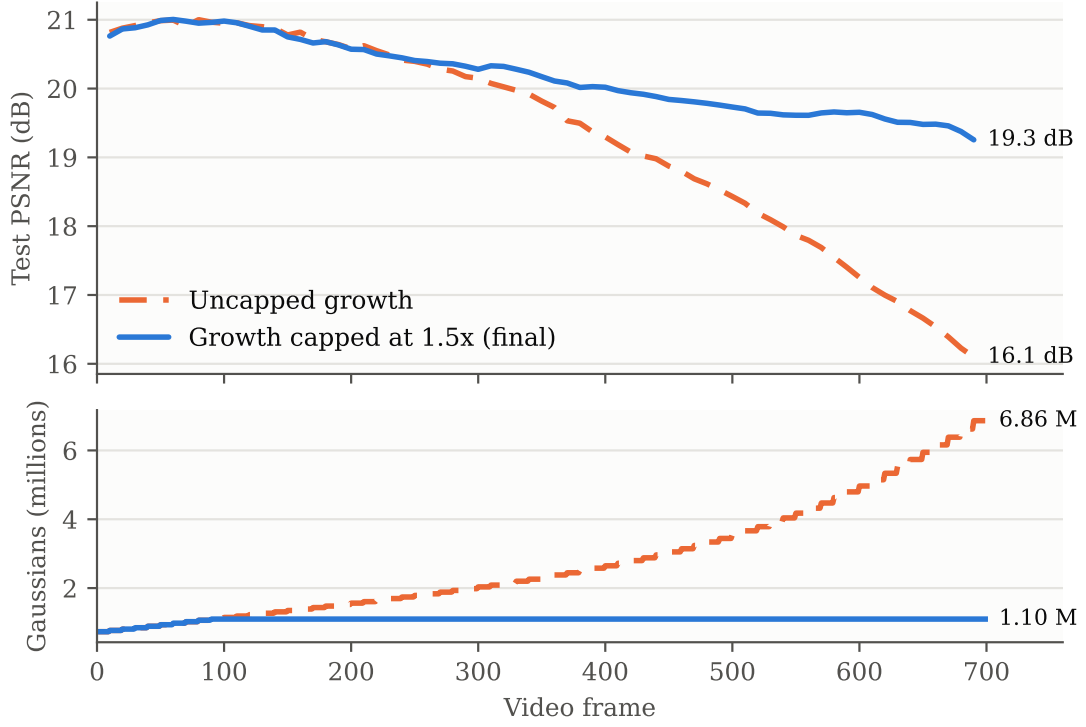


Figure 2: Test PSNR (top) and scene size (bottom) over the video, before and after the growth cap. **Setup:** RTX PRO 6000 (96 GB); training and test images at 1920×1080 (the 4K capture halved, then undistorted); all 700 frames. Each point is the average over the 24 test cameras.



Figure 3: Before the fix, frame 650: the best, median, and worst test cameras (left: real image, right: render). The worst view is covered by a large dark floater. **Setup:** RTX PRO 6000 (96 GB); training and test images at 1920×1080 (the 4K capture halved, then undistorted); all 700 frames.

5.2 What the test cameras see

Figure 4 shows the final method at an early frame (100) and a late, crowded frame (650). To avoid hand-picking flattering views, a script scores all 24 test cameras at that frame and shows the best, the median, and the worst. At frame 100 even the worst view is close to the real image. At frame 650 the court and crowd are still clean, but players in the crowded play near the far basket become blurred or broken up, and a few thin floaters appear. This matches the numbers: the background holds up, while player quality drops in the second half (23.85 to 21.69 dB).



Figure 4: Final method: best, median, and worst test cameras at frames 100 and 650 (left: real image, right: render). **Setup:** RTX PRO 6000 (96 GB); training and test images at 1920×1080 (the 4K capture halved, then undistorted); all 700 frames.

5.3 Results at native 4K

[4K 700-frame run in progress; results, videos, and the 1080p versus 4K comparison are added when it finishes.]

5.4 Earlier development results

Table 5 records how the method improved during development on the RTX 3080, before the final PRO 6000 runs. These runs used a slightly different undistorted image size, so their numbers are close to, but not directly comparable with, Table 4.

Table 5: Development history. **Setup:** all rows RTX 3080 (12 GB); training images 1874×1054 and test images 1879×1055 (the 4K capture halved, then undistorted); scores are averages over the 24 test cameras unless the last column says otherwise. Test frames are every tenth video frame, which are also keyframes.

Method	Full PSNR	Person PSNR	Person IoU	Measurement
Global 3DGS baseline	17.50	—	—	frame-0 test
Stage A canonical	19.32	18.34	0.788	frame-0 test
Stage B v1	18.29	15.00	0.70→0.30	keyframe test mean
Stage B v2 (old)	18.87	19.62	0.71→0.39	keyframe test mean
Focused refinement, frames 1–20	22.42	26.76	0.746	training-view mean
300-frame diagnostic	20.07	22.44	—	test mean



Ground truth, view 13, frame 200



Earlier Stage B v2



CourtFlow-GS diagnostic

Figure 5: The same test camera (view 13) at frame 200 during development. **Setup:** RTX 3080 (12 GB); training images 1874×1054 and test images 1879×1055 (the 4K capture halved, then undistorted).

5.5 Why these design choices were selected

The method was not chosen by adding complexity for its own sake. Table 6 records the practical failures we observed and the smallest change that addressed each one. These are useful trade-offs: some choices improve quality and speed at the same time by avoiding work on irrelevant pixels, while others deliberately spend more computation only where player detail needs it.

Table 6: Design choices and the observation behind each one. **Setup:** observations from development runs on the RTX 3080 (12 GB); training images 1874×1054 and test images 1879×1055 (the 4K capture halved, then undistorted); the last row comes from the final RTX PRO 6000 runs at 1920×1080 .

Choice	Why it was selected
3D-first identities	Matching 2D centres and colours independently fragmented 68 masks into 60 apparent people. Building rough 3D components first gives the same player one ID across cameras.
Occlusion-aware hull	A strict silhouette intersection removed hidden limbs. Allowing arbitrary misses inflated a hull by about $2.5\times$. Measured nearer depth supplies the middle ground.
Masked MAST3R crops	Full-frame descriptors reduce a small player to about 6×25 pixels and produced star-shaped rays. Crops spend the fixed descriptor resolution on the player instead of the court.
Same-group MLS controls	A global control graph lets nearby teammates pull each other. Fixed same-player control neighbours keep motions separate and also make deformation cheap.
Shared translation plus local offsets	Moving every control alone reached 0.60 IoU by frame 10; shared translation plus local articulation reached 0.71 IoU.
Crop refinement plus keyframes	Full-image updates mostly optimize static court pixels. Player crops improve moving detail cheaply; full keyframes occasionally correct accumulated error and add splats.
Growth cap (found on the PRO 6000)	Uncapped keyframe growth compounded to 6.86M Gaussians and 16.1 dB by frame 699. Capping the total at $1.5\times$ the frame-0 size kept 1.10M Gaussians and 19.3 dB, and was faster.

5.6 Evaluation protocol and deliverables

The 12 training cameras are never used to judge quality. At every tenth frame and at the last frame, the tracker renders all 24 test cameras and compares them with the real images from those cameras. Nothing about the method was tuned on the test cameras except the growth cap, which was chosen from a short 300-frame comparison; its effect on the full video is reported above.

The run writes two animated videos while it tracks: a test-camera video (view 13) and a camera that orbits the court. Each video frame is drawn from the scene exactly as it was at that moment of the game. The final results folder also keeps the editable frame-0 scene (79 MB), the control-point paths for all 700 frames (109 MB), and the final scene state (114 MB, 1.10 million Gaussians). A single saved state is one instant of the game, not the whole video: storing every frame naively would take about $700 \times 114 \text{ MB} \approx 80 \text{ GB}$, so a practical package would store the frame-0 scene plus the per-frame motion in compressed form.

6 Runtime

6.1 The full 700-frame run

Table 7 shows where the time goes in one complete run of the final method on the RTX PRO 6000, from the raw videos to finished novel-view videos.

Table 7: End-to-end time for the final method. **Setup:** RTX PRO 6000 (96 GB); training and test images at 1920×1080 (the 4K capture halved, then undistorted); all 700 frames. The tracking row is the final (growth-capped) run; the steps before it are identical in both 700-frame runs and were reused from the first run.

Step	Wall-clock time
Prepare input (undistort frame 0)	0.3 min
Stage A: build and score the frame-0 scene	9.9 min
Decode 700 frames from 36 videos	11.2 min
SAM2 player masks, 12 cameras one after another	31.2 min
Track the ball	10.7 min
Track players and refine, 700 frames (incl. test scoring and videos)	48.3 min
Total	≈ 112 min

Tracking itself costs 3.71 s per video frame (0.27 frames per second): 1.36 s moving control points, 1.09 s refining player crops on nine frames out of ten, 6.49 s for each tenth-frame keyframe, and 0.72 s (averaged) for masks and identity repair. Peak GPU memory was 7.2 GB. Almost half of the time before tracking is spent on data preparation rather than reconstruction; Section 5.3 measures the faster data path (JPEG frames, GPU undistortion, and SAM2 on four cameras at once) used for the 4K run.

Novel-view rendering (inference). Drawing a tracked frame from a new camera takes about 2–3 ms on the RTX PRO 6000, even at 4K; Section 7 covers inference in detail.

6.2 RTX 3080 versus RTX PRO 6000

Table 8: The same work on both GPUs. **Setup:** RTX 3080 (12 GB) with training images at 1874×1054 and tracking renders at 937×527 ; RTX PRO 6000 (96 GB) with training images at 1920×1080 and tracking renders at 960×540 . The PRO 6000 handles 5% more pixels, so its speed-ups are slightly understated. Both GPUs ran the same settings, including the 3D identity repair. The Stage A row is the whole frame-0 build. The Stage B rows average video frames 2–29 (28 frames, two keyframes) on both GPUs; frame 1 is left out because the 3080 spent 159 s compiling its CUDA code there, which the PRO 6000 does ahead of time. The raster row draws the *same* saved 3080 scene (2.0 million Gaussians) on both GPUs at 937×527 .

Component	RTX 3080 12 GB	RTX PRO 6000	Speed-up
Stage A frame-0 canonical build	1896 s	528 s	$3.6\times$
MLS deformation/control optimization	2.75 s/frame	1.28 s/frame	$2.1\times$
of which fused MLS forward	0.072 s/frame	0.029 s/frame	$2.5\times$
Focused crop refinement (100 iterations)	3.04 s/non-keyframe	0.92 s/non-keyframe	$3.3\times$
500-step full keyframe	23.02 s/keyframe	5.75 s/keyframe	$4.0\times$
Mask refresh with identity repair, amortized	1.83 s/frame	1.02 s/frame	$1.8\times$
Stage B total	9.05 s/frame (0.110 FPS)	3.57 s/frame (0.280 FPS)	$2.5\times$
Colour + raster of a saved scene, 937×527	4.36 ms (229 FPS)	2.20 ms (455 FPS)	$2.0\times$

The fused MLS/SVD calculation is only a small part of tracking time; drawing the Gaussians and computing gradients through that drawing dominate. The PRO 6000 gains most where the work is large: keyframes are $4.0\times$ faster and crop refinement $3.3\times$. Moving control points gains less ($2.1\times$); each of its 50 steps draws only four small, half-resolution cameras, which likely leaves much of the larger GPU idle. Mask refresh gains least ($1.8\times$) because part of the identity repair runs on the CPU.

7 Inference: showing the reconstruction to a viewer

Reconstruction (Stages A and B) is the slow, one-off part. Showing the result needs no training and none of the neural networks used earlier (SAM2, MAST3R, the person detector). To draw one moment of the game from any camera, only two steps run on the GPU:

1. **Bake** the moment: move every Gaussian to where it is at that frame. The control points store the motion; one fused CUDA kernel applies it (the same MLS step used during tracking).
2. **Draw**: evaluate each Gaussian’s colour for the viewing direction and rasterize (splat) the Gaussians into the image.

Table 9: Inference cost per frame. **Setup:** RTX PRO 6000 (96 GB), idle GPU, CUDA-synchronized timing averaged over 300 frames (drawing) or 20 repeats (baking); video encoding excluded.

Step	Scene	Time per frame
Bake (move all Gaussians with 15,352 control points)	2.0M Gaussians	0.9–2.2 ms
Draw at 1920×1080	1.10M Gaussians	2.20 ms (455 FPS)
Draw at 3840×2160	1.10M Gaussians	2.94 ms (341 FPS)
Draw at 1920×1080 and 3840×2160	2.0M Gaussians	[pending]

Baking plus drawing therefore takes about 3–5 ms, which is well above real time: the video itself runs at 25 frames per second (40 ms per frame).

7.1 What has to be stored

A saved scene is one instant of the game. To play the whole video, the simplest option is to store a full scene for every frame, but at 2 million Gaussians that is about 200 MB per frame in our current PLY format, or 140 GB for 700 frames. It is tempting to store the scene once and then only the control-point positions for each frame, because the control points carry the motion. We tested this directly: starting from the saved scene at frame 600 and moving it with the stored control points, every player lands within 0.013 scene units of where the full run placed it at frame 650 (players had moved up to 11 units). The motion is therefore captured almost exactly. The picture is not: the full run also updates colours and adds detail at every keyframe and refines players every frame, and without those updates the test-camera PSNR is 0.6 dB lower after 10 frames and 1.4 dB lower after 50.

A faithful and compact package therefore stores three things:

- the background once, plus the few background changes made at keyframes;
- the players’ Gaussians at every keyframe (every 10 frames; about 193,000 Gaussians, under 10% of the scene);
- the control-point positions for every frame (15,352 points, about 92 kB per frame at half precision), which move the players between keyframes.

The Gaussians themselves can be made much smaller. Our PLY files spend about 104 bytes per Gaussian on full-precision numbers and unused fields. Half precision alone halves that. Vector quantization goes further: instead of storing every Gaussian’s scale, rotation and colour, it stores a small “codebook” of typical values and, per Gaussian, the index of the closest one. Published methods for static scenes report 15–50 $\times$ smaller files at almost the same quality (LightGaussian [8], CompGS [7]); QUEEN applies the same idea to per-frame changes in video [3]. We have not yet compressed our scenes, so the following numbers are *estimates* from

those published ratios: roughly 5–10 MB for the background, under 1 MB per keyframe of players, and the control points, which is about 10–35 Mbit/s depending on how well the control motion is compressed. That is the same range as a single 4K video stream.

7.2 Two ways to serve viewers

Delivering this to many people is closer to streaming a video game than to running a neural network, and the two standard game-streaming designs both apply.

Render on the server (like cloud gaming). The viewer sends where their virtual camera is; a server GPU bakes and draws that view, encodes it with the GPU’s video encoder, and streams ordinary video back. Any device can watch, and the bandwidth is that of normal video. The costs are a network round trip whenever the viewer moves the camera (tens of milliseconds) and GPU time per viewer. Three things make this efficient at scale: bake each moment once and share it between all viewers watching that moment (baking is per game frame, drawing is per viewer); draw several viewers’ cameras in one rasterizer call; and hand finished frames straight from GPU memory to the video encoder. Our current exporter instead copies every frame to the CPU and pipes it to ffmpeg, which is now its slowest step.

Render on the viewer’s device (like downloading game assets). The server only sends the compressed package above, and the viewer’s GPU bakes and draws. Camera moves are instant and server cost is little more than a content-delivery network, but the viewer needs a capable GPU: 2 million Gaussians is heavy for a phone. The usual fix from games is level of detail: send fewer, larger Gaussians for distant or less important parts of the scene. A service would likely use both, with server rendering for weak devices and local rendering for strong ones.

7.3 Why “just compile it” does not work here

Tools such as TensorRT speed up neural networks by optimizing a fixed graph of large, regular operations (matrix multiplications, convolutions) whose sizes are known in advance. Gaussian splatting is not like that. Every frame it projects the Gaussians, sorts them by depth within each small screen tile, and blends them front to back, and how many Gaussians land in each tile changes with every camera and every moment of the game. The work is dominated by sorting and irregular memory access in custom CUDA kernels, which such compilers cannot express or improve. Speed-ups therefore come from engineering the renderer itself, as game engines do:

- **Fuse steps.** Baking is already a single kernel; folding the colour evaluation into it would remove another pass over all Gaussians.
- **Remove launch overhead** by recording the whole bake-and-draw sequence once as a CUDA graph and replaying it every frame.
- **Reuse work across frames.** Consecutive frames are nearly identical, so the previous depth order is an excellent starting point for the next sort.
- **Skip what is not visible:** drop Gaussians outside the camera’s view before sorting, and use coarser detail far away.
- **Port the renderer** to Vulkan or WebGPU compute shaders so it runs in game engines and web browsers, where client-side viewing would happen.

8 Can the method use fewer cameras?

We did not rebuild the full pipeline with fewer than 12 cameras, but two measurements indicate where the limit is. First, when one training camera was removed (leaving a 60° gap), player PSNR seen from that gap fell from 18.3 to 17.0 dB, so a single wider gap already costs about 1.3 dB. Second, the method finds each player in 3D by asking at least four cameras to agree. With 12 cameras (30° apart) most players are seen by enough cameras; with 6 (60° apart) many players would be seen well by only two or three, and identities would start to fail. Our estimate is that 9 cameras (40° apart) is the practical minimum for this design, at a cost of roughly 1 dB, while 6 would need a stronger shape prior, for example a learned human body model, to replace the missing views.

9 Limitations and next steps

- **Players in crowded plays.** Player tracking still degrades late in the video. The training-camera mask overlap falls from 0.55 at frame 100 to about 0.20 after frame 600, and the method needed 190 recoveries, where a lost player is reset from its frame-0 shape. Player PSNR drops from 23.85 dB in the first half to 21.69 dB in the second. The next step is to reduce these recoveries, for example by tracking players jointly when they overlap instead of one at a time.
- **A few thin floaters.** Some long, thin Gaussians near the cameras still show up as streaks in a few test views. Limiting how elongated a new Gaussian may become would target them.
- **Data preparation time.** Decoding and masking take almost as long as tracking at 1080p. The faster data path used for 4K should be made the default at every resolution.
- **Storage.** A compact package (frame-0 scene plus compressed per-frame motion) is needed to ship the whole video instead of one state per frame.

References

- [1] J. Luiten, G. Kopanas, B. Leibe, D. Ramanan. Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis. 3DV 2024. <https://arxiv.org/abs/2308.09713>
- [2] J. Sun et al. 3DGStream: On-the-Fly Training of 3D Gaussians for Efficient Streaming of Photo-Realistic Free-Viewpoint Videos. CVPR 2024. <https://arxiv.org/abs/2403.01444>
- [3] S. Girish et al. QUEEN: QUantized Efficient ENcoding of Dynamic Gaussians for Streaming Free-viewpoint Videos. NeurIPS 2024. <https://arxiv.org/abs/2412.04469>
- [4] J. Liang et al. ClipGStream: Clip-Stream Gaussian Splatting for Any Length and Any Motion Multi-View Dynamic Scene Reconstruction. CVPR 2026. <https://arxiv.org/abs/2604.13746>
- [5] Y.-H. Huang et al. SC-GS: Sparse-Controlled Gaussian Splatting for Editable Dynamic Scenes. CVPR 2024. <https://arxiv.org/abs/2312.14937>
- [6] H. Park, G. Ryu, W. Kim. DropGaussian: Structural Regularization for Sparse-view Gaussian Splatting. CVPR 2025. <https://arxiv.org/abs/2504.00773>
- [7] K. L. Navaneet et al. CompGS: Smaller and Faster Gaussian Splatting with Vector Quantization. ECCV 2024. <https://arxiv.org/abs/2311.18159>
- [8] Z. Fan et al. LightGaussian: Unbounded 3D Gaussian Compression with 15x Reduction and 200+ FPS. NeurIPS 2024. <https://arxiv.org/abs/2311.17245>